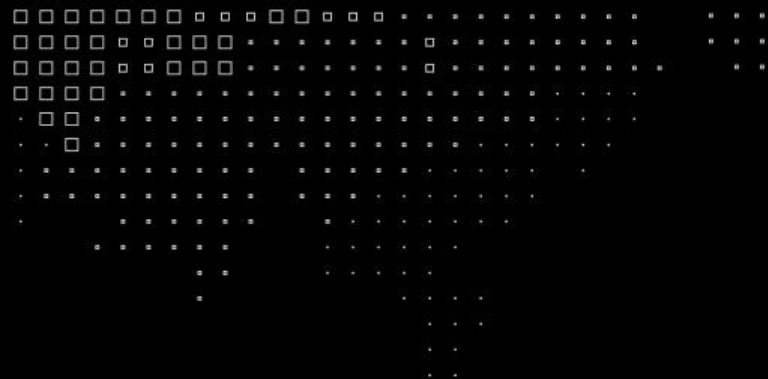
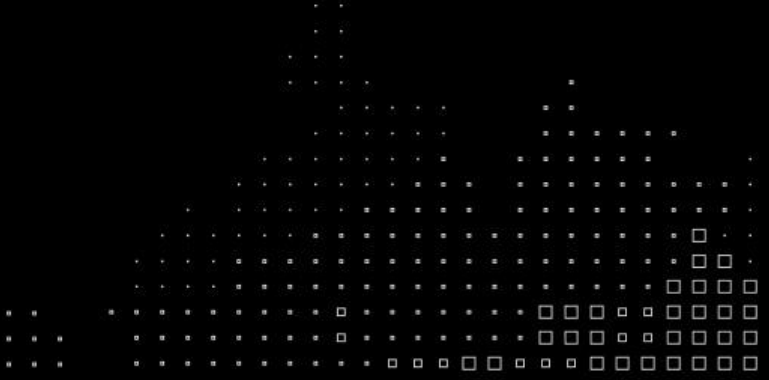
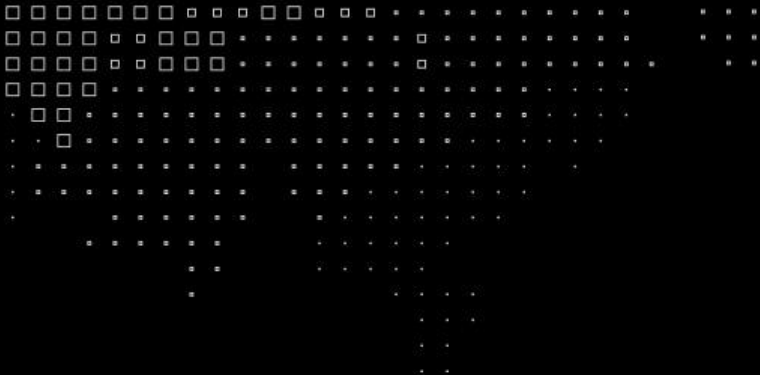




MBA⁺

AI Engineering &
Multi-Agents

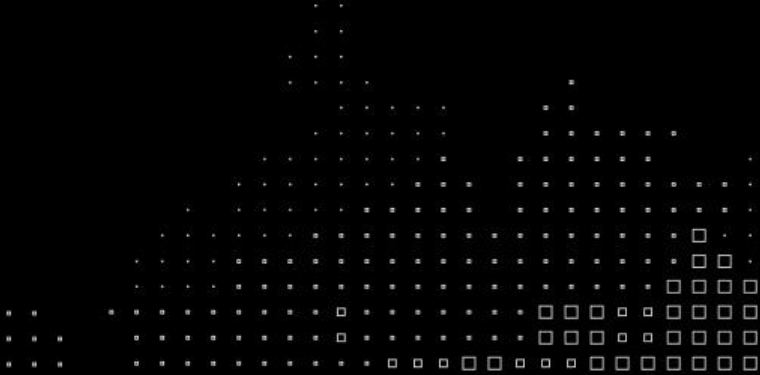




MBA⁺

Data Architecture

Prof.: Leandro Mendes



BANCOS VETORIAIS PARA AGENTES

Operação Q —

O cérebro do agente (LLM) já veio configurado de fábrica. O sistema de memória e conhecimento dele é você quem sobe e modela hoje.

Prof. Leandro



O que nós vamos construir e dominar hoje?

1

Infraestrutura e Modelagem

Subir o pgvector e projetar o schema de dados ideal para o agente (políticas estáveis vs. memórias dinâmicas de conversas).

2

Operações de Runtime & Segurança

Buscas por similaridade semântica com proteção contra vazamento de dados entre clientes (isolamento transacional).

3

Engenharia de Busca & Índices

Indexar coleções em larga escala (HNSW vs. IVFFlat) pesando custo de memória, tempo de build, latência (p95) e precisão (recall).

4

Tomada de Decisão Tecnológica

Comparar Postgres + pgvector com soluções embarcadas (FAISS) e bancos nativos (Pinecone, Weaviate, Qdrant).

BLOCO A ·
conceitos 35'



prática guiada
A 20'



Atividade 1 ·
60'



debrief 5'

BLOCO B ·
indexação 30'



prática guiada
B 15'



Atividade 2 ·
60'



fechamento 15'

Entrega: arquivo .zip com os notebooks e fichas de decisão por grupo, no sistema da FIAP.

O nosso laboratório: Quantum Finance e o Agente Q

A EMPRESA

Quantum Finance — banco digital em plena expansão, com 2 milhões de clientes ativos e 59 manuais de procedimentos internos (tarifas, produtos, compliance).

O AGENTE Q

Assistente automatizado para atender clientes. Seu loop de decisão está pronto e selado (chassi_agente.py): lê ferramentas e responde citando documentos. Nosso papel é fornecer a base de dados que ele consulta.



O AVALIADOR

Script Python robusto e determinístico (sem LLM de julgamento) que simula o comportamento dos clientes e valida se o banco construído por vocês atende aos requisitos de segurança e eficiência.



Marina · cli-001

economizando para uma moto · NÃO quer ofertas de cartão

Carlos · cli-002

foco absoluto em investimentos para previdência

Ana · cli-003

renegociou a dívida · vai exercer o direito de esquecimento (LGPD)

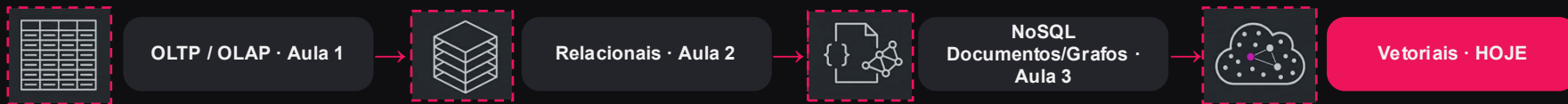
Bruno · cli-004

cliente novo · não pode, sob hipótese alguma, ver o histórico da Marina

Fernanda · cli-005

atendimento urgente: sinistro do seguro celular

Bancos Relacionais vs. Bancos Vetoriais: onde mora o significado?



SEM BANCO VETORIAL

O agente busca palavras exatas ("saque"). Se o cliente pergunta "quanto me cobram para tirar dinheiro?", o relacional não encontra correspondência e o agente alucina: "Bancos cobram entre R\$ 5 e R\$ 10..."

COM BANCO VETORIAL

O agente busca por proximidade semântica ("tirar dinheiro" se aproxima de "tarifas de saque") e recupera a informação real: "A tarifa é de R\$ 7,50 (doc prod-tabela-tarifas)."

FRONTEIRAS DE APRENDIZADO

Hoje (Arquitetura): subir o banco, definir o schema, metadados, filtros, escrita em runtime, isolamento transacional, exclusão definitiva e indexação em RAM.

Amanhã (Knowledge Management & Prompt): modelos de embedding, chunking semântico, buscas híbridas (BM25 + vetores), frameworks de orquestração e avaliações de RAG.

O que é um vetor na prática?

ESCALAR

Um valor numérico único, sem direção.
temperatura de hoje = 31 °C · tarifa = R\$ 7,50 · idade = 31 anos
Em programação: um float ou int.

VETOR

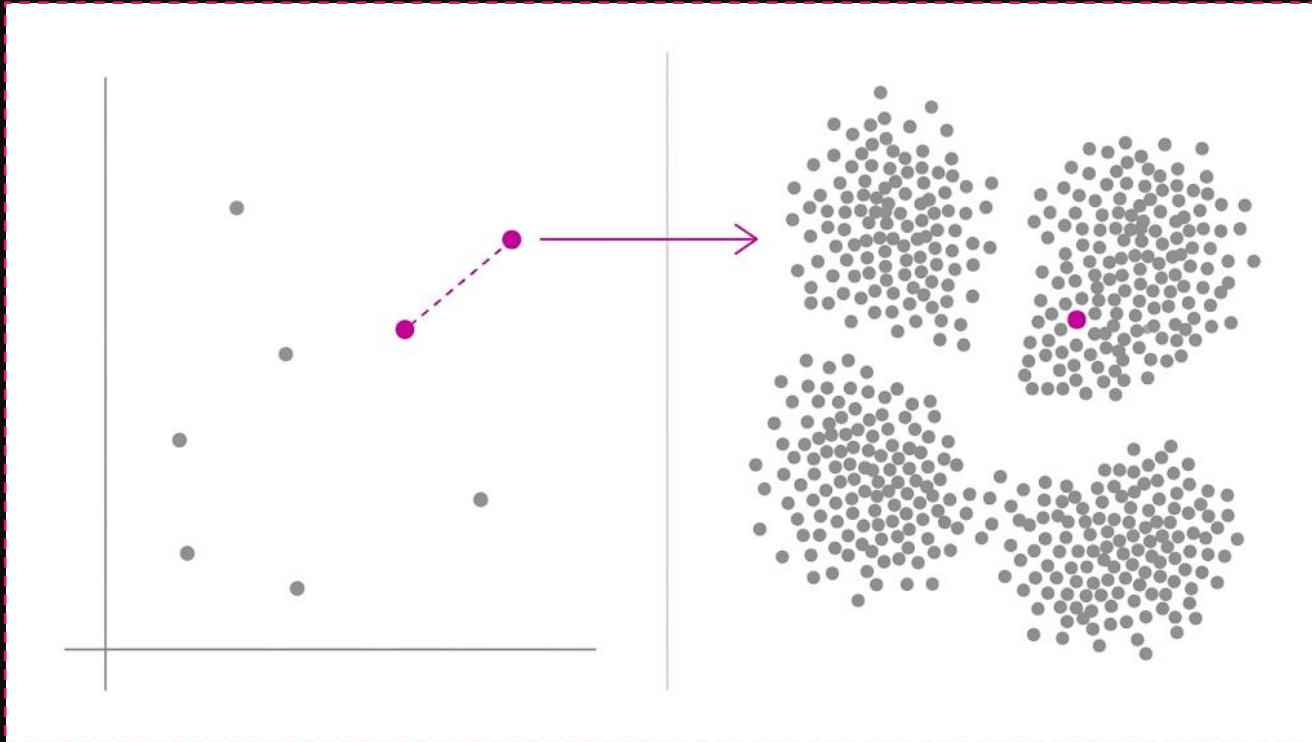
Lista estruturada de números que descrevem um elemento. A ordem das posições é rígida.
Clima: (temp_min, temp_media, temp_max) = (17, 24, 31)

PERFIL DE CLIENTE

(renda_mensal, idade, produtos_ativos, score)
Marina = [7000.00, 31.0, 2.0, 610.0]
Carlos = [12000.00, 48.0, 5.0, 790.0]



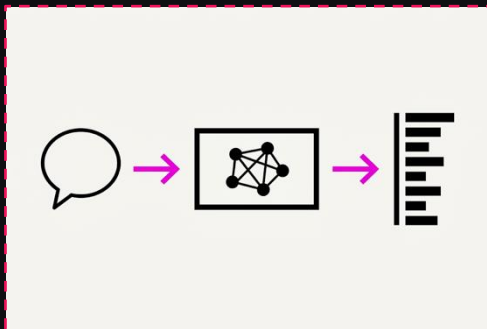
Ao converter propriedades em coordenadas numéricas, comparar dois clientes vira uma tarefa matemática: calcular a distância entre dois pontos no espaço.



Duas dimensões dá para desenhar. Trezentas e oitenta e quatro, não — mas a matemática de distância é exatamente a mesma.

Traduzindo palavras para coordenadas: o que é um embedding?

Representação numérica de um texto (palavra, frase ou parágrafo) gerada por um modelo de rede neural congelado, onde textos com significados semelhantes são mapeados para coordenadas muito próximas no espaço matemático.



```
"meu notebook não liga"      → [ 0.12, -0.08,  0.31, ...]  
"computador não inicializa"  → [ 0.11, -0.07,  0.29, ...]  
    ← extremamente próximos, sem palavras iguais!  
"quanto rende o CDB?"       → [-0.22,  0.40, -0.05, ...]  
    ← distantes: o significado é outro
```

AS 3 REGRAS DE INFRAESTRUTURA DO EMBEDDING

1 · Exatamente o mesmo modelo na ingestão e na consulta · 2 · O nº de dimensões do modelo define a coluna: vector(384) · 3 · Mudou de modelo? Recalcular e reindexar toda a base.

No laboratório: paraphrase-multilingual-MiniLM-L12-v2 (384 dimensões, processamento local).

A busca do banco vetorial: quem são os meus vizinhos semânticos?

θ

DISTÂNCIA DE COSSENO $\leq$

- 0 = mesma direção (significado idêntico)
- 1 = ortogonais (nenhuma relação)
- 2 = opostos (significados invertidos)
- ignora o tamanho físico do texto

```
SELECT doc_id, titulo, texto
FROM conhecimento
ORDER BY embedding <=> :query_vector
LIMIT k;
-- <=> = 1 - similaridade de cosseno · a query central do k-NN
```

O CUSTO LINEAR DE BUSCA (o desafio de escala)

Para poucas dezenas de linhas, a busca linear exata (Seq Scan) é instantânea. Mas com 40 milhões de memórias de clientes, o banco teria de fazer 15 bilhões de cálculos de cosseno por consulta. Para evitar que o sistema caia, precisamos de índices aproximados (ANN) — Bloco B.



Uma imagem, um número: o exato escala linearmente. O agente não pode esperar segundos por resposta.

A Escada da Maturidade de Dados para IA

Regra: subir de degrau antes da hora gera desperdício de dinheiro e operação. Demorar para subir causa gargalos, vazamentos e paradas.

1 · EMBEDDING EM MEMÓRIA

Numpy / listas

- até ~50 mil vetores, cargas estáveis e estáticas

Uso real: classificador de intenções no chat (triagem de e-mails de SAC); roteamento simples de tarefas

2 · RAG EMBARCADO

Chroma local, FAISS

- busca semântica sobre documentos locais, em um único servidor/processo

Uso real: assistente sobre os manuais de RH; copiloto de código no repositório local

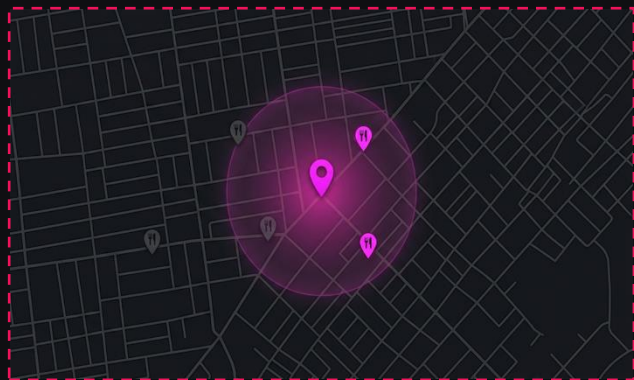
3 · BANCO DE PRODUÇÃO

pgvector, bancos nativos

- milhões de vetores · concorrência de escrita e leitura em tempo real
- governança (LGPD, backup ACID, RBAC) · isolamento por cliente

Uso real: sistema de memória ativa dos 2 milhões de correntistas da Quantum

Anatomia de um Banco Vetorial



A ANALOGIA ESPACIAL

"Ache os restaurantes italianos mais próximos de mim (proximidade física), que aceitem reservas (filtro estruturado)."

O banco vetorial faz o mesmo em 384 dimensões: "ache as memórias parecidas com a dúvida do cliente (proximidade semântica), filtrando só as do cliente autenticado cli-001 (filtro estruturado)".

ID	Vetor (384d)	Metadados (tipo, data, cliente_id)	Documento original
mem-17	[0.12, -0.08, ...]	cli-001 · 2026-07-09	"Marina pediu para NÃO receber ofertas de cartão"

O QUE O BANCO VETORIAL NÃO É

Não substitui o relacional (transações e joins de faturamento seguem no OLTP) · Não é um grafo (Neo4j): a vizinhança é calculada por distância em runtime, não por relações fixas de nós e arestas · Não tem "compreensão": calcula distâncias estritas sobre as coordenadas que a sua aplicação gravou

PostgreSQL + pgvector: integrando vetores ao mundo relacional

```
CREATE EXTENSION vector;                -- ativação da extensão

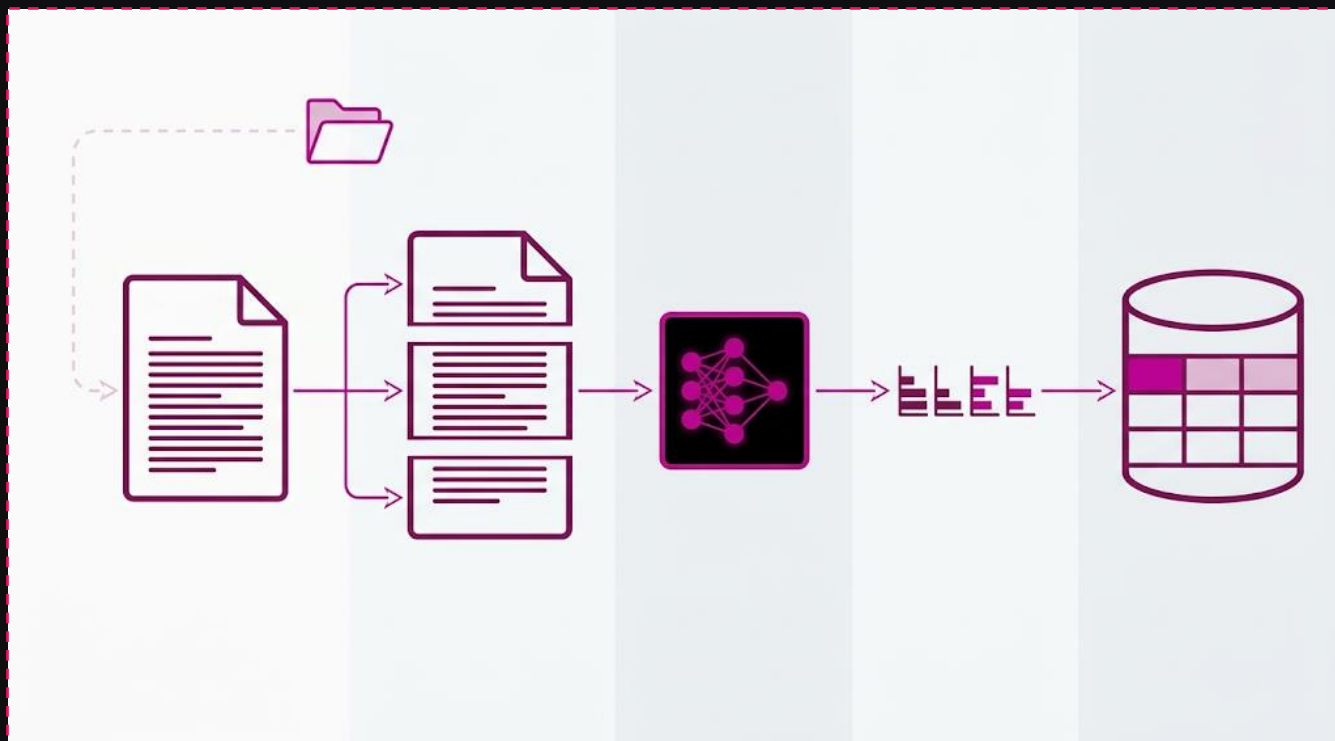
CREATE TABLE conhecimento (
  id text PRIMARY KEY, doc_id text, titulo text, tipo text, data date, texto text,
  embedding vector(384)                -- dimensões rígidas!
);

INSERT INTO conhecimento VALUES ('tarifa#3', 'prod-tarifas', 'Tabela de Tarifas', 'oficial',
  '2026-03-01', 'Saques custam R$ 7,50.', '[0.12, -0.08, 0.31, ...]');
-- o vetor é calculado pela APLICAÇÃO e inserido como array estruturado

SELECT doc_id, titulo, texto FROM conhecimento
WHERE tipo = 'oficial' ORDER BY embedding <=> :q_vector LIMIT 3;  -- consulta com filtro
```

VANTAGEM UNIFICADORA

Você aproveita a consistência ACID, joins com o cadastro de clientes, transações seguras, regras de backup e o RBAC do Postgres que sua empresa já opera.



Ninguém guarda um PDF "em vetor": extrai-se o texto, fatia-se, vetoriza-se cada pedaço e grava-se uma linha por pedaço.

Modelando o schema do agente: conhecimento vs. memória

TABELA conhecimento · carga offline, só leitura pelo agente

- Origem: manuais oficiais de políticas, FAQs autorizados, tabelas vigentes
- Acesso: o agente busca trechos semânticos filtrando por documentos oficiais (WHERE tipo = 'oficial')

TABELA memória · leitura e escrita dinâmica em runtime

- Origem: interações em tempo real que o agente decide gravar (preferências, status de conversas)
- Acesso: escrita imediata pós-atendimento · busca restrita ao cliente autenticado (WHERE cliente_id = :autenticado)

1

Isolamento estrito via banco: filtro SQL no código das ferramentas de dados. Nunca delegue o isolamento ao prompt do LLM!

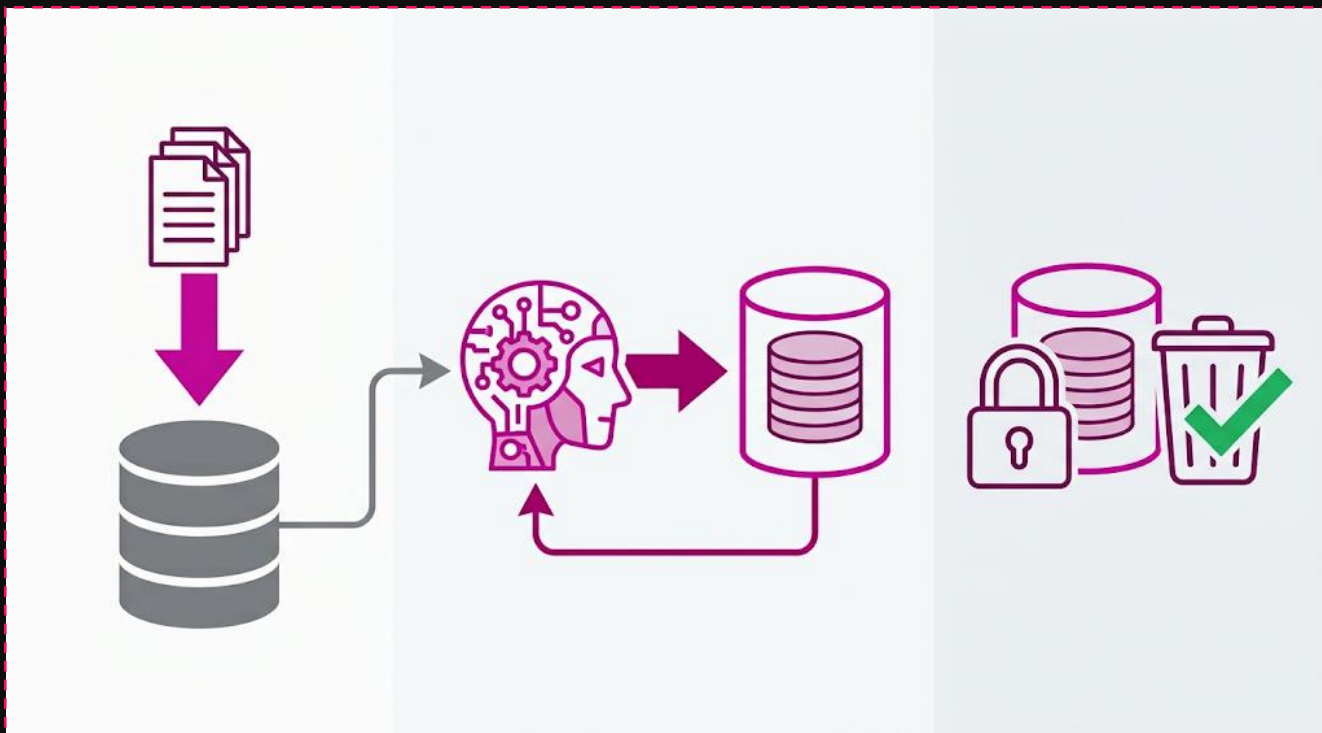
2

Exclusão transacional (LGPD real): DELETE físico real. Soft delete (flag ativo = false) mantém o dado e viola a conformidade.

3

Chaves estruturadas: una a memória ao cadastro tradicional com restrições de chave estrangeira.

O SCHEMA DO AGENTE · QUEM ESCRIVE ONDE



Duas tabelas, dois padrões de acesso: só a ingestão escreve no conhecimento; o próprio agente escreve na memória — e o WHERE é a porta.

O que é chunking e por que fatiamos documentos?

A NECESSIDADE FÍSICA

Um PDF ou .docx inteiro não vira um único vetor útil. O modelo de embedding tem limites de entrada e o agente precisa de parágrafos focados para ler e citar — não de livros inteiros.

CHUNK = UMA LINHA DE BANCO

Fatiamos o documento em segmentos discretos. Cada bloco vira um registro separado, com seu próprio vetor e referências ao manual de origem.

documento .md



chunk 1 · linha 1



chunk 2 · linha 2



chunk 3 · linha 3

O MÉTODO DO KIT (`utils.chunk_paragrafo`)

Fatiamento por parágrafos reais (linhas em branco), adicionando o cabeçalho ao bloco: "Tabela de Tarifas 2026 — [texto do parágrafo]".

Estratégias profundas de fatiamento semântico, tamanhos ideais e Parent-Document: Knowledge Management & Prompt.

Transição para o laboratório: Atividade Prática 1

ONDE

Google Colab.

A primeira célula do notebook compila e sobe uma instância limpa de Postgres + pgvector dentro da VM efêmera em menos de 3 minutos.

O FLUXO DE TRABALHO

- Projetar o DDL das tabelas
- Executar a esteira de ingestão dos documentos estruturados
- Criar as funções de consulta semântica com filtros
- Acoplar as funções como ferramentas do Agente Q

O AVALIADOR · 10 CHECKS

- Schema: tipos de colunas e dimensionalidade exata
- Ingestão: contagens e completude
- Semântica: documento esperado para queries-chave
- Segurança: o Bruno nunca enxerga a Marina
- LGPD: esquecer() apaga fisicamente

 vazamento entre clientes = nota travada em 40

Demonstração guiada: integrando o pgvector ao Agente Q

1

Rodar o setup e carregar a extensão.

2

Executar o DDL de criação das tabelas.

3

Carga em lote dos dados usando COPY binário.

4

Busca semântica livre (o FAQ de 2023 é retornado indevidamente) → o filtro tipo = 'oficial' corrige o ranking sem mexer na rede neural.

5

Simular o vazamento ao buscar memórias sem filtro por ID → correção robusta com WHERE cliente_id.

6

Plugar a ferramenta no chassi do Agente Q.

```
def buscar_conhecimento(query):  
    """Busca na base oficial (apenas tipo = 'oficial')."""  
    v = emb([query])[0]  
    with conn.cursor() as cur:  
        cur.execute("""SELECT doc_id, titulo, texto FROM conhecimento  
                        WHERE tipo = 'oficial'  
                        ORDER BY embedding <=> %s LIMIT 3""", (v,))  
        rows = cur.fetchall()  
        return "\n\n".join(f"[fonte: {d}] {t}\n{x}" for d, t, x in rows)  
  
q_agente = ChassiAgente()  
q_agente.registrar_ferramenta(buscar_conhecimento,  
                                descricao="Busca políticas, tarifas e regras oficiais...",  
                                parametros={"query": {"type": "string", "description": "..."}})
```

Atividade Prática 1: a memória protegida da Quantum Finance

1 · CENÁRIO

59 manuais estruturados em corpus/ (.md com metadados no cabeçalho: área, tipo, data e título; 40 políticas vigentes e 19 ruídos históricos) e 24 interações passadas de clientes em dados/conversas_seed.json, com históricos e preferências.

2 · O DESAFIO

O Q está ativo, mas sem banco inventa tarifas e não lembra de nada. Modelar as duas tabelas no pgvector para que o agente (i) encontre e cite exclusivamente os manuais vigentes e oficiais; (ii) lembre do histórico do cliente autenticado com segurança absoluta; (iii) apague permanentemente os dados de quem exercer a LGPD.

3 · REGRAS DE ARQUITETURA

Embedding fixo: MiniLM de 384 dimensões · chassi_agente.py travado · decisões do grupo: D1 design de tabelas e tipagem de metadados · D2 estratégia de filtros das queries · D3 isolamento transacional de clientes · D4 (opcional) ordenação combinada por recência.

4 · ENTREGÁVEIS

Notebook executado com saídas · resultado_a1.json gerado pelo script validador · Ficha de Decisão preenchida.

Consolidando aprendizados: o que a Atividade 1 nos ensina?

RODADA DE EXPERIÊNCIAS

Quem quer compartilhar seu score e qual decisão de design de schema mais pesou no resultado do avaliador?

LIÇÃO 1 · O PODER DO SQL

Não mexemos em uma linha da rede neural nem no prompt. O agente ficou confiável, seguro contra vazamentos e apto a citar fontes porque desenhamos colunas corretas, um WHERE inteligente e um DELETE físico em conformidade.

LIÇÃO 2 · O ALERTA INVISÍVEL

O EXPLAIN mostrou um Seq Scan. Com 300 parágrafos, menos de um milissegundo. Com 40 milhões de memórias, o agente para de responder pela latência.

→ Próximo passo: Bloco B, indexação avançada.

Busca vetorial em escala: o bibliotecário sem catálogo



BUSCA LINEAR EXATA (sem índice)

Você entra numa biblioteca gigante e lê a lombada de absolutamente todos os volumes. Garantia absoluta de achar a melhor correspondência — mas inviável em escala.

BUSCA APROXIMADA (com índice ANN)

Um bibliotecário experiente entende o assunto e te leva direto a um corredor. Segundos. Mas um livro guardado por engano em outra seção pode ficar de fora.

RECALL@k

Dos k vizinhos verdadeiros (busca exata), quantos o índice retornou? 9 de $10 = \text{recall } 0,90$. Medida direta de perda de precisão.

p95

Ordene 100 consultas; a 95ª é o p95. "p95 ≤ 20 ms" garante que 95 em 100 usuários esperam no máximo 20 ms.

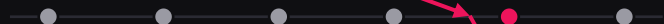
Índice HNSW: grafos de proximidade em camadas

Hierarchical Navigable Small World — cada vetor vira um nó ligado aos seus m vizinhos mais próximos. Camadas superiores: pouquíssimos nós, ligações longas ("estradas expressas"). Camadas inferiores: alta densidade, conexões curtas ("ruas de bairro"). A busca começa veloz pelo topo, dá saltos longos em direção à query e desce de camada para refinar.

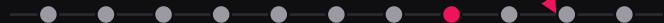
camada 2 · avião



camada 1 · táxi



camada 0 · a pé



SP → capital (avião) → cidade vizinha (táxi) → nº da casa (a pé)

m

ligações máximas por nó · mais precisão (recall), muito mais RAM · padrão 16

ef_construction

varredura no build · grafo melhor, build mais lento · padrão 64

ef_search

candidatos por query · AJUSTÁVEL POR SESSÃO em runtime, sem reconstruir · padrão 40

O CUSTO FÍSICO: o grafo mora 100% na RAM

$RAM \approx N \times (d \times 4 B + m \times 2 \times 8 B)$

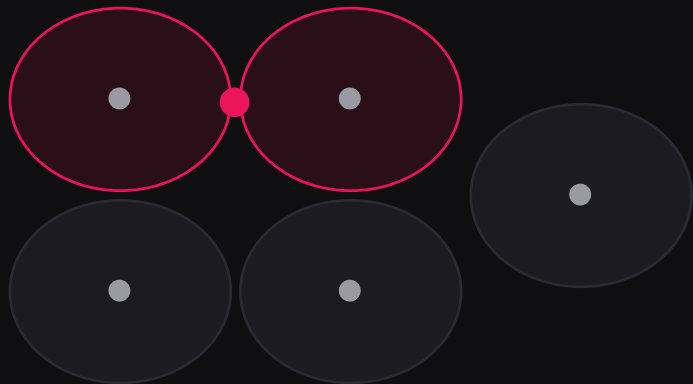
Medido (100k × 384d): build 83 s com maintenance_work_mem = '1GB' · índice 195 MB · consulta 7 ms (Seq Scan: 90 ms)



Avião, táxi, a pé: saltos longos no topo, passos curtos na base. $\sim \log N$ passos em vez de N .

Índice IVFFlat e técnicas de quantização de vetores

Inverted File Index — no build, um k-means fatia o espaço em lists (clusters/bairros) e calcula o centróide de cada um. Na consulta, a query acha os probes centróides mais próximos e limita a busca linear aos vetores desses bairros. Analogia: em vez de listar todos os restaurantes do país, você olha só as 2 zonas gastronômicas mais próximas.



query na fronteira · probes = 2 → o vizinho no 3º bairro fica de fora

PARÂMETROS E TRADE-OFF

- lists: $N/1000$ até 1 M linhas, $\sqrt{N}$ acima · probes: $\sqrt{\text{lists}}$ de partida (mais = recall ↑, CPU ↑)
- Prós: build em 3,5 s (100k, Colab), RAM adicional insignificante
- Contras: recall degrada sob atualizações intensas; criar DEPOIS da carga inicial

QUANTIZAÇÃO — compactação de vetores

- halfvec (float16): metade da RAM, recall virtualmente idêntico
- bit (binária): até 32× menor · pré-filtro por Hamming <-> → 50 candidatos → re-rank exato



O ecossistema vetorial: bibliotecas, extensões e bancos nativos



FAISS (Meta)

biblioteca in-process

C++/Python de alta performance, roda localmente no seu processo. Motor puro de busca vetorial em CPU/GPU. Sem servidor, sem integridade de tabelas, sem concorrência nativa de escrita.



pgvector

extensão do Postgres

Integra HNSW e IVFFlat diretamente ao Postgres: queries SQL unificadas e consistência ACID transacional imediata.



Pinecone

SaaS gerenciado

Nuvem totalmente gerenciada. Zero gerenciamento de infra, escalabilidade automatizada, namespaces isolados de fábrica.



Weaviate

SGBD nativo (OSS)

Alta performance. Busca híbrida nativa (BM25 + vetores) out-of-the-box, módulos de vetorização acopláveis, compressões de RAM.



Qdrant

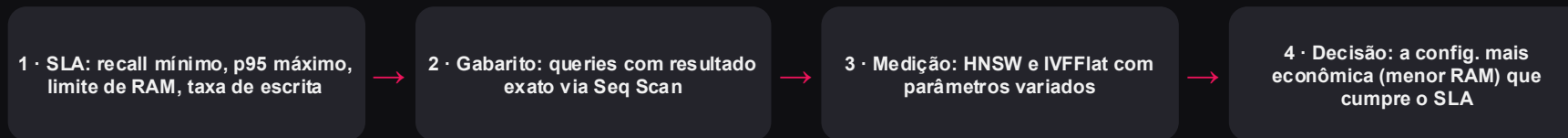
SGBD nativo (Rust)

Motor extremamente flexível de filtros sobre payloads JSON ricos e ampla variedade de quantizações nativas configuráveis.

Matriz de decisão arquitetural e métodos de engenharia

Critérios	pgvector	FAISS (lib)	Pinecone	Weaviate	Qdrant
Natureza	extensão de Postgres	biblioteca local	SaaS em nuvem	SGBD nativo	SGBD nativo
Filtro por metadado	SQL completo	manual	sim	sim	sim (payload)
Escrita / ACID	sim (rígida)	não	eventual	sim	sim
Busca híbrida	lógica SQL manual	não	sim	sim (nativa)	sim (vetores sparse)
Operação	DBA tradicional	toda sua	nenhuma infra (SaaS)	média (self-host)	média (self-host)
Consumo de RAM	alto (HNSW em RAM)	alto	pago por uso	alto	otimizado

O método científico de escolha do índice





Cada milissegundo do p95 é uma pessoa esperando do outro lado do chat.

Atividade Prática 2: dimensionando a escala da Quantum Finance

A Quantum aprovou o Agente Q para produção. A base de memórias dinâmicas cresceu para 100 mil vetores de 5 mil clientes ativos simulados. Três departamentos definiram SLAs distintos para o uso da MESMA tabela de dados vetoriais:

A · ATENDIMENTO AO VIVO (Operações)

O cliente está ativamente conversando no chat com o agente.

p95 ≤ 20 ms · recall@10 ≥ 0,95 · inserções intensas o dia todo

B · AUDITORIA DE COMPLIANCE (Jurídico)

Varredura em lote de madrugada em servidores compartilhados e pequenos.

recall@10 ≥ 0,99 sem exceções · tempo irrelevante · índice ≤ 100 MB de RAM

C · VISÃO INDIVIDUAL (Atendimento)

Consultas focadas estritamente em um único cliente por vez (WHERE cliente_id = :user_id).

recall@10 ≥ 0,90 com filtros seletivos · p95 ≤ 50 ms

O DATASET DE TESTE

100 mil memórias sintéticas de 384 dimensões e um gabarito de 200 queries com os vizinhos exatos pré-calculados de forma linear. O avaliador mede por cenário: recall@10 · p95 · tamanho do índice · tempo de build · SLA cumprido? · o mais barato entre os que cumprem.

Ciclo de medição e sintonia fina de índices

1

Gerar 100k vetores e carregar em lote em < 3 s via COPY.

2

medir('exato'): busca linear → recall = 1,0 · p95 ≈ 90 ms · RAM extra = 0 MB.

3

SET maintenance_work_mem = '1GB' → HNSW padrão (m=16, ef_construction=64) em ~83 s → 195 MB → medir('hnsw'): recall ≈ 0,93 · p95 ≈ 7 ms.

4

Ajuste em runtime: SET hnsw.ef_search = 100, sem reconstruir → recall ≈ 0,98 · p95 ≈ 12 ms.

5

Filtro seletivo: busca de um único cliente → só 3 resultados (o Postgres filtrou depois do grafo) → SET hnsw.iterative_scan = relaxed_order → 10 resultados.

```
SET maintenance_work_mem = '1GB';  
SET hnsw.ef_search = 100;  
SET hnsw.iterative_scan = relaxed_order;  
-- comandos de runtime: por sessão, sem  
rebuild
```

configuração	recall@10	p95	tamanho	build
exato (Seq Scan)				
hnsw m16/ef64				
hnsw + ef_search 100				
hnsw + iterative_scan				

Atividade Prática 2: sintonia e dimensionamento de índices

1 · CENÁRIO

A Quantum precisa que vocês decidam e dimensionem o motor vetorial para a base ampliada. O kit gerou no Colab o arquivo de 100 mil memórias estruturadas de clientes e as queries de validação.

2 · O PROBLEMA

Encontrar, por experimentação real, as configurações de índice (HNSW ou IVFFlat) e os parâmetros de build e runtime que atendem aos SLAs de cada departamento pelo menor custo operacional (menor tamanho em disco/RAM).

3 · DIRETRIZES TÉCNICAS RÍGIDAS

SET `maintenance_work_mem` = '1GB' obrigatório antes de criar índices · no mínimo duas configurações empíricas por cenário · decisões: D1 tipo de índice por departamento · D2 parâmetros de build (`m`, `ef_construction` ou `lists`) · D3 parâmetros de runtime (`ef_search` ou `probes`; `iterative_scan`) · D4 uso estratégico de quantizações (`halfvec`).

4 · ENTREGÁVEIS

Notebook executado com as queries de build e os logs de medições · `resultado_a2.json` · Ficha de Decisões Técnicas preenchida, incluindo a projeção teórica para 40 milhões de memórias.

Rubrica (50 pts): Avaliador Automático 30 (10 por cenário: aderência ao SLA e menor tamanho de índice) | Ficha de Decisões Técnicas 15 | Parâmetros de runtime corretos 5 🕒 58 min

Desafio de alta performance: otimizações extremas em escala

1 · COMPRESSÃO EXTREMA COM RE-RANKING

Indexação binária com `binary_quantize(embedding)`. Busca em duas etapas: (i) top-50 candidatos ultrarrápidos pela distância de Hamming (`<~>`); (ii) re-rank exato desses 50 pela distância de cosseno real. Medir economia de RAM e latência.

2 · CENÁRIO C EM LARGA ESCALA

Comparar a eficiência do `ef_search` ampliado vs. particionamento declarativo de tabelas por faixas de `cliente_id` no Postgres.

REGRAS DE ENTREGA

Notebook bônus com a tabela comparativa de testes preenchida e um parágrafo fundamentado de arquitetura para a escala de 40 milhões de memórias de correntistas.

★ Bônus de até 10 pontos extras aplicados diretamente na média da disciplina

FECHAMENTO

Vocês dominaram a engenharia de memória dos agentes!

BLOCO A

Dominamos a infraestrutura do pgvector: schemas seguros para agentes, isolamento rigoroso no SQL, conformidade LGPD com exclusões transacionais e bases oficiais acopladas ao Agente Q em runtime.

BLOCO B

Entendemos a física da busca vetorial em escala: sintonia de HNSW e IVFFlat, quantizações, parâmetros ajustados em tempo real para responder a múltiplos SLAs corporativos.

NO MERCADO

Sistemas financeiros (histórico + crédito ao lado do cadastro) · busca de e-commerce (milhões de SKUs + filtros de estoque) · bases de conhecimento multimodais para vários robôs.

Bancos vetoriais não são brinquedos isolados de IA: são o sistema de memória persistente dos agentes — e projetar essa memória é engenharia e arquitetura de dados.



Instruções de entrega e critérios de avaliação

NO .ZIP DO GRUPO

- atividade1_pgvector.ipynb e atividade2_indices.ipynb totalmente executados (com todas as saídas)
- resultado_a1.json e resultado_a2.json gerados pelos avaliadores
- FICHA_DECISAO.md com as seções das duas atividades preenchidas
- (opcional) desafio_quantizacao.ipynb
- Nome: grupoNN_operacaoQ.zip · um representante entrega, todos os nomes na ficha

Item	Pts	Critério
A1 · avaliador	30	$\geq 95 \rightarrow 30$ · $\geq 85 \rightarrow 24$ · $\geq 70 \rightarrow 18$ · vazamento $\rightarrow$ máx 12
A1 · ficha	15	decisões D1–D4 5 · evidência numérica 5 · "40 milhões" 5
A1 · Q citando fontes	5	cita doc_id 5 · sem citar 2 · não executado 0
A2 · avaliador de SLAs	30	por cenário: SLA cumprido 6 + índice mais barato 4
A2 · ficha de escala	15	tabela ≥ 2 config. 5 · justificativas 5 · RAM p/ 40M 5
A2 · parâmetros de runtime	5	ef_search / probes / iterative_scan explicados
Bônus · desafio	10	tabela 6 · parágrafo 4

Penalidades: mesmo/a Arquiteto/a nas duas atividades –5 · notebook sem saídas –10 por notebook · chassi alterado $\rightarrow$ zera o critério de citação